

1. Qu'est-ce que GitLab CI/CD ?

GitLab CI/CD est un outil intégré à GitLab qui permet d'automatiser :

- **CI (Continuous Integration / Intégration Continue)** : Tester et vérifier automatiquement votre code à chaque push ou merge.
- **CD (Continuous Deployment / Continuous Delivery)** : Déployer automatiquement votre application dans un environnement de test ou de production.

L'automatisation se fait grâce à un fichier appelé **.gitlab-ci.yml** placé à la racine de votre projet.

2. À quoi sert le fichier **.gitlab-ci.yml** ?

Ce fichier décrit :

1. **Les étapes (stages)** : phases par lesquelles votre pipeline va passer (ex: install, test, build, deploy).
2. **Les jobs** : actions concrètes qui s'exécutent à chaque stage.
3. **Les runners** : agents qui exécutent vos jobs.

Il permet de dire à GitLab exactement **quoi faire et dans quel ordre** pour automatiser votre CI/CD.

3. Structure idéale pour une application React

Pour une application React, on peut imaginer 4 étapes :

1. **Install** : installer les dépendances (npm install)
2. **Lint** : vérifier la qualité du code (eslint)
3. **Test** : exécuter les tests (npm test)
4. **Build** : créer la version prête pour la production (npm run build)
5. **Deploy** : optionnel, déployer sur un serveur ou un service comme Netlify, Vercel ou GitLab Pages.

4. Exemple complet de **.gitlab-ci.yml** pour React

```
# Nom du fichier : .gitlab-ci.yml
# Définit le pipeline CI/CD pour une application React

stages:
  - install    # étape 1 : installation des dépendances
  - lint       # étape 2 : vérification de code
  - test       # étape 3 : tests unitaires
  - build      # étape 4 : construction du projet
  - deploy     # étape 5 : déploiement (optionnel)
```

Job pour installer les dépendances

install_dependencies:

stage: install # ce job appartient à l'étape "install"
image: node:18 # image Docker contenant Node.js
script:
- npm install # commande pour installer les packages
cache:
 paths:
 - node_modules/ # dossier à mettre en cache

Job pour vérifier la qualité du code avec ESLint

lint_code:

stage: lint
image: node:18
script:
- npm install
- npx eslint src/**
allow_failure: false # si ce job échoue, le pipeline échoue

Job pour exécuter les tests

test_app:

stage: test
image: node:18
script:
- npm install
- npm test -- --watchAll=false
artifacts:
 when: always # conserve les résultats même si le job échoue
 paths:
 - coverage/ # dossier contenant le rapport de couverture de tests

Job pour construire l'application

build_app:

stage: build
image: node:18
script:
- npm install
- npm run build
artifacts:
 paths:
 - build/ # le dossier build sera disponible après le job

Job de déploiement (exemple sur GitLab Pages)

deploy_app:

stage: deploy
image: node:18
script:
- npm install
- npm run build
artifacts:
 paths:
 - public/

only: - main	# ce job ne se déclenche que sur la branche "main"
-----------------	--

5. Explication détaillée des termes

Terme	Rôle / Explication
stages	Définit les différentes phases du pipeline. Les jobs s'exécutent dans cet ordre.
stage	Attribue un job à une étape spécifique.
image	Image Docker utilisée pour exécuter le job (ici Node.js pour React).
script	Liste de commandes shell exécutées pour ce job.
cache	Permet de stocker des dossiers pour accélérer le pipeline (ex: node_modules).
artifacts	Fichiers ou dossiers à conserver après l'exécution du job (ex: build, coverage).
allow_failure	Si vrai, le job peut échouer sans arrêter le pipeline.
only	Déclenche le job seulement pour certaines branches ou tags.